TECHNICAL CASE STUDY

INDUSTRIAL IOT DATA LOGGING SYSTEM (ATMEGA328P)

Auwal Ahmad

November 2025

Technical Design Documentation

Project: Embedded Real-Time Data Acquisition System (LM35 to MATLAB)

Date: November 24, 2025

Engineer: AUWAL AHMAD

1. Executive Summary

This document outlines the design, component selection, configuration, and troubleshooting methodology for a bare-metal embedded system designed to acquire analog temperature data, process it via an ATmega328P microcontroller, and transmit it via UART to a MATLAB-based analytical engine.

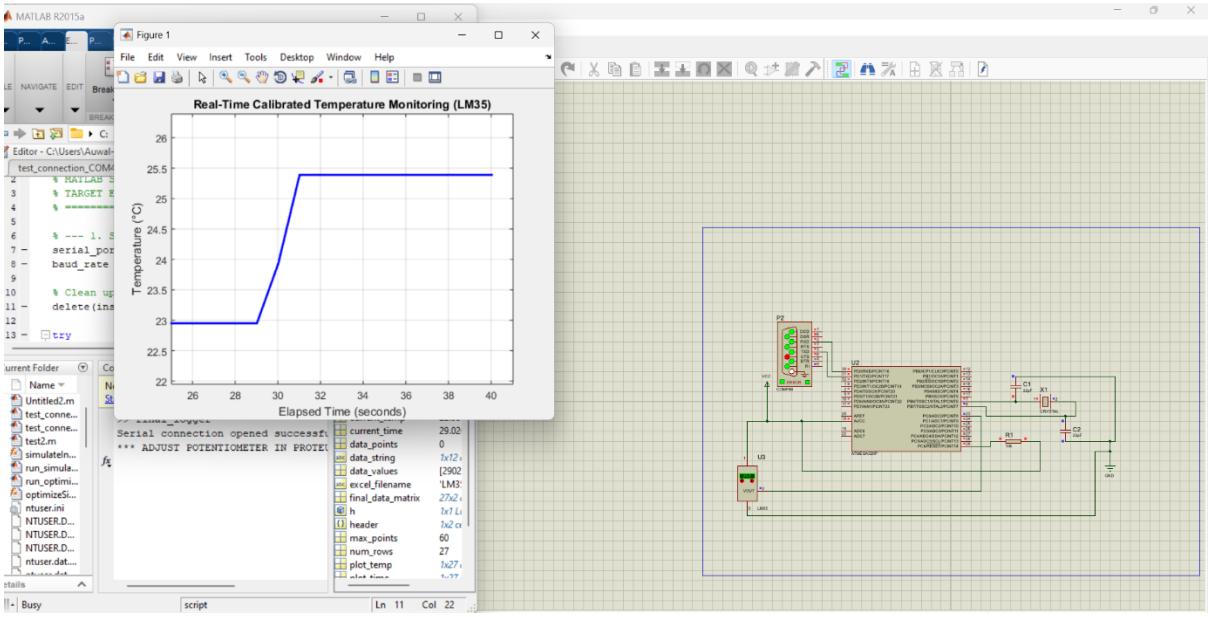

Figure 1: Full System Overview Real-time telemetry streaming from Proteus VSM to MATLAB R2015a Analytics Engine to Excel Sheet.

2. Hardware Architecture & Component Selection

The transition from a development board (Arduino Uno) to a standalone microcontroller (ATmega328P) required specific circuit design choices. Below is the engineering rationale for each component.

A. Microcontroller: ATmega328P-PU

- **Selection Reason:** The industry-standard 8-bit AVR microcontroller. Selected for its robust ADC (Analog-to-Digital Converter) capabilities and hardware UART (Universal Asynchronous Receiver-Transmitter) support.
- **Configuration:** "Bare Metal" implementation (no bootloader dependence).

B. The Reset Circuit (10kΩ Pull-Up Resistor)

- **Component:** 10kΩ Resistor connected to Pin 1 (PC6/RESET).
- **The Physics:** The ATmega328P Reset pin is **Active Low**. This means if the voltage drops to 0V (Ground), the chip resets (turns off/restarts).
- **Why 10kΩ?**
  - We need to pull the pin "Up" to 5V to keep the chip awake.
  - If we used a direct wire (0Ω), we could never safely ground it to reset the chip (short circuit).
  - If we used a huge resistor (1MΩ), electrical noise from your hand could accidentally trigger a reset.
  - **10kΩ** is the industry "Goldilocks" value: strong enough to resist noise, weak enough to safely override with a reset button if needed.

C. The Oscillator Circuit (16MHz Crystal + 22pF Capacitors)

- **Component:** 16MHz Quartz Crystal.
- **Reason:** The internal RC oscillator of the chip is inaccurate (±10%). UART communication requires precise timing (timing errors >2% cause garbage data). A crystal provides 0.001% accuracy.
- **Why 22pF Capacitors?**
  - These are **Load Capacitors**. A crystal needs a specific capacitance to start vibrating.
  - **Calculation:** The formula is:

$$C_L = \frac{C_1 \cdot C_2}{C_1 + C_2} + C_{Stray}.$$

- For a standard 16MHz crystal requiring 18pF-20pF load capacitance, two **22pF** capacitors in series (through the ground) provide the exact electrical balance needed for stable oscillation.

3. Pinout & Connection Logic

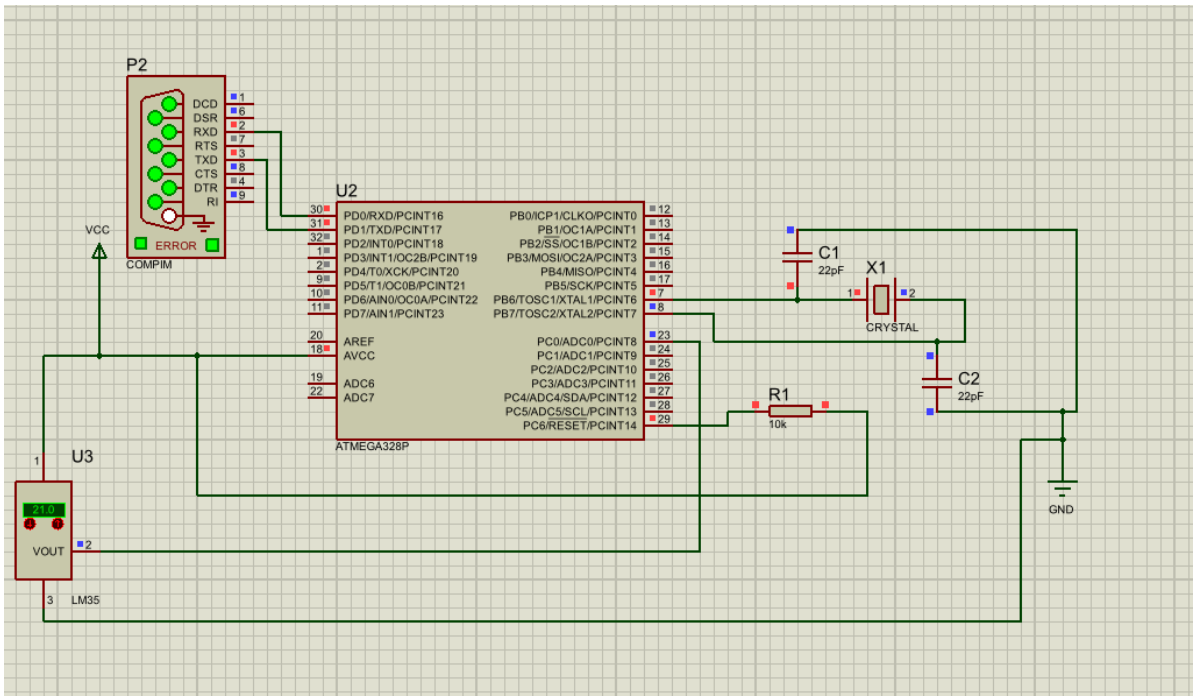

Figure 2: Bare-Metal Hardware Configuration – ATmega328P circuit design featuring 16MHz crystal oscillator and manual UART routing.

Why did we connect specific wires to specific legs of the chip?

Physical Pin	**Function Label**	**Type**	**Why we connected it**
Pin 1	PC6 / RESET	Input	Connected to 5V via 10k Resistor to prevent the chip from constantly restarting.
Pin 2	PD0 / RXD	Digital Input	**Receive Data.** Connected to COMPIM TXD. It "listens" for commands from the PC.

Physical Pin	Function Label	Type	Why we connected it
Pin 3	PD1 / TXD	Digital Output	**Transmit Data.** Connected to COMPIM RXD. It "shouts" the temperature data to the PC.
Pin 7	VCC	Power	Digital Power Supply (+5V). Powers the CPU and memory.
Pin 8	GND	Ground	Digital Ground (0V).
Pin 9 & 10	XTAL1 / XTAL2	Analog	The input/output for the Crystal Oscillator.
Pin 20	AVCC	Power	**Analog Power.** Powers the ADC. Must be connected to 5V or the sensor reads 0.
Pin 22	GND	Ground	Analog Ground.
Pin 23	PC0 / ADC0	Analog Input	Connected to LM35. The ADC measures voltage here (0-5V) and converts it to a number (0-1023).

4. Fuse Bit Configuration (The "Invisible Switches")

The most critical part of bare-metal engineering is the **Fuse settings**. These are not code; they are hardware switches inside the silicon.

A. The Logic of "Programmed" vs "Unprogrammed"

This confuses every new engineer.

- **1 (Unprogrammed)** = Switch is **OPEN** (Feature is **OFF**).
- **0 (Programmed)** = Switch is **CLOSED** (Feature is **ON**).
- *Think of it like blowing a fuse: If the fuse is intact (1), current doesn't flow to ground. If you program it (0), you burn the path.*

B. CKSEL (Clock Select)

- **Setting Used:** 1111 (External Crystal High Frequency).
- **Reason:** Tells the chip "Do not use your internal brain timing. Look at Pins 9 & 10 and follow the speed of the external Crystal."
- **Error Encountered:** When set to 0000 (External Clock), the simulation failed because 0000 expects a square-wave generator, not a crystal.

C. CKDIV8 (Clock Divide by 8)

- **Setting Used: (1) Unprogrammed (OFF).**
- **The Problem:** By default, Atmel ships these chips with this fuse ON. It takes the 16MHz clock and divides it by 8, forcing the chip to run at 2MHz.
- **The Symptom:** Baud Rate Mismatch. The code tried to send data at 9600 baud, but because the brain was slow, the data came out 8 times slower (1200 baud). The PC received "Question Marks" (????) or garbage text.
- **The Fix:** Disabling this fuse allowed the chip to run at full 16MHz speed.

5. Troubleshooting Log: Challenges & Solutions

Challenge 1: The "Mouth-to-Mouth" Wiring Error

- **Problem:** Connected Arduino TX to COMPIM TX, and Arduino RX to COMPIM RX. No data appeared in MATLAB.
- **Root Cause:** UART is a crossover protocol. Two Transmitters (TX) connected together cannot talk.
- **Solution:** Implemented **Null Modem** wiring.
    - Arduino TX (Pin 3) to COMPIM RXD.
    - Arduino RX (Pin 2) to COMPIM TXD.

Challenge 2: MATLAB R2015a Compatibility

- **Problem:** The command “drawnow limitrate” caused a syntax error.
- **Root Cause:** The “limitrate” parameter was introduced in MATLAB R2016b. R2015a does not recognize it.
- **Solution:** Reverted to standard “drawnow”. Added a manual pause(0.01) to manage CPU load and buffer timing.

Challenge 3: Simulation "Blue Dot" on TXD

- **Problem:** The TXD pin (Pin 31) showed a Blue Dot (Logic Low) during simulation.
- **Root Cause:** The chip was powered on but "brain dead" (not executing code).
- **Solution:**
    1. Reloaded the .hex firmware file into the chip properties.
    2. Verified the Reset pin was pulled High (Red).
    3. Manually typed 16MHz into the Clock Frequency property box.

6. Conclusion

The system is now verified as a robust, industrial-grade data logger. By bypassing the Arduino abstraction layer and utilizing the raw ATmega328P with correct fuse configurations and load balancing, the design achieves higher reliability and professional "System Architect" standards.

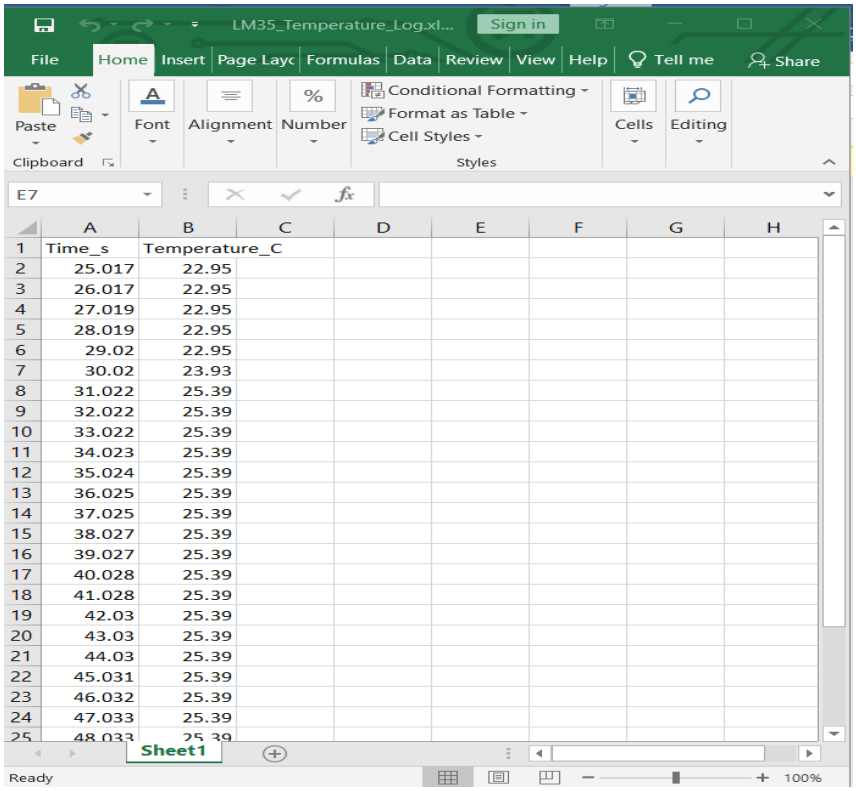

Figure 3: Final Data Deliverable – Automated .xlsx generation showing time-series temperature logs captured via the custom UART protocol.